Sonder

A GenAI Travel-Planning and Co-Traveller-Matching System

---

1. Abstract

Sonder is a generative-AI travel product that takes a user from a blank screen to a personalised, day-by-day itinerary, then matches them with compatible co-travellers and lets the pair negotiate the actual trip together in real time.

The product addresses three intertwined problems that traditional travel sites do not solve coherently:

- **Specificity at scale.** Algorithmic recommenders surface "things to do in Lisbon"; what travellers actually want is *"the rosé-coloured cafés on Largo do Carmo at golden hour"*. We use retrieval-augmented generation grounded in a curated venue corpus to write itineraries that read like a friend's recommendation rather than a TripAdvisor list.
- **Compatibility, not popularity.** Co-traveller matching is usually a free-text bio plus a swipe. Sonder runs a transparent, multi-feature compatibility pipeline over a vector store of candidate travellers, with hard safety filters and per-user learning from in-conversation signals.
- **Believability of the social layer at cold start.** A travel app with no users feels dead. Sonder seeds a population of LLM-designed synthetic travellers who autonomously post, open trips for companions, and message real users about their plans — sustaining a live social surface from day one.

The implementation is end-to-end GenAI with light agentic orchestration: a multi-provider language-model routing layer (small tier for fast persona voice, large tier for itinerary planning, dedicated validator tier for quality control), a three-namespace vector index for destinations / activities / travellers, real-time chat over websockets with sub-two-second persona replies, a streamed itinerary-generation pipeline that lets users see day one within fifteen seconds, and a validator engine that catches assistant-voice leakage, hallucinated venues, and broken persona consistency before they reach the user.

Measured outcomes: itineraries with near-zero negative-constraint violations versus 10-25% on a prompt-only baseline; persona-grounded chat replies in 1-2 seconds at one-twenty-fourth the cost of a large-model baseline; a co-traveller matching system that produces calibrated compatibility scores driving honest reciprocal-approval rates rather than uniformly high cosine numbers.

---

2. Exploratory Data Analysis

2.1 What data Sonder operates over

Sonder is not a classical ML system with a single static training set. It is a live product that produces and consumes data across four planes:

Plane	What it holds	Why it matters
Curated reference corpus	~50 destinations and ~500 activities, each with an embedded text context	Grounding for retrieval-augmented itinerary generation
Synthetic traveller population	192 LLM-designed solo travellers (96 per gender) and 18 couples	The matching pool, social-layer content authors, and conversation partners
Operational user data	Profiles, generated itineraries, chat sessions and messages, shared-itinerary negotiation history, journal entries, social feed posts, join requests	The product's runtime state
Telemetry	Validator outcomes, retrieval counts, match scores, latency distributions, hallucination flags	Product observability — every LLM surface is instrumented

2.2 Foundation datasets and theoretical frameworks

Sonder's intelligence layer is built on three foundation datasets and two academic frameworks that we incorporated rather than inventing:

GoEmotions — fine-grained emotion taxonomy. Source: *Demszky et al. (2020), "GoEmotions: A Dataset of Fine-Grained Emotions"* (arXiv:2005.00547). The original dataset contains 58,000 Reddit comments annotated across 27 emotion labels plus a neutral category. We do not ship the 58,000 labeled training rows. Instead, we use the 27-label vocabulary as anchor vectors: each label's tone-anchored gloss is embedded once at process start using the same 1536-dimensional embedding model that powers the rest of the persona pipeline, and a user's free-text input is classified by cosine similarity against those 27 anchor vectors. This gives us defensible emotion scores that live in the same embedding space as everything else, without depending on a separately trained classifier model that would need its own retraining cycle. Twenty-seven labels include *admiration, amusement, anger, annoyance, approval, caring, confusion, curiosity, desire, disappointment, disapproval, disgust, embarrassment, excitement, fear, gratitude, grief, joy, love, nervousness, optimism, pride, realization, relief, remorse, sadness, surprise*. Glosses are deliberately tone-anchored, not dictionary-like — for example, *realization* is *"a quiet click — coming to understand something"* rather than a denotative definition. This phrasing choice matters because it crisps the embedding space at the cost of being slightly idiosyncratic.

Push-Pull Motivation Theory — travel psychology framework. The 12-dimension persona space is grounded in *Push-Pull Motivation Theory*, originally introduced in travel research by Dann (1977) and extended by Crompton (1979). The theory holds that travel decisions are driven by two distinct motivational forces:

- **Push factors** (intrinsic, traveller-side): why a person leaves home — the internal psychological pulls toward travel itself.

- **Pull factors** (extrinsic, destination-side): what attributes of a destination they seek — what about the place draws them specifically.

Our implementation defines six push dimensions and six pull dimensions, all unipolar (positive presence only — bipolar concepts like introvert vs. extrovert sociality emerge compositionally across multiple dimensions rather than via dedicated negative axes):

Push dimensions (why)	Pull dimensions (what)
escape_reset — disconnect, recharge, leave routine	*nature_outdoors* — landscapes, weather, physical settings
adventure_novelty — push themselves, first-time experiences	*culture_history* — heritage, museums, local arts
connection — share with people they love	*food_drink* — local cuisine, regional specificities
reflection — process, gain perspective	*nightlife_social* — bars, clubs, live music
curiosity — understand, go deeper than guidebook	*comfort_luxury* — high-end stays, refined service
prestige_reward — milestone trips, dream destinations	*exploration_local* — neighbourhoods, daily-life immersion

Each dimension is defined by a substring-matched keyword set rather than a single trigger word. Multi-word phrases like *"out of comfort zone"* (mapped to *adventure_novelty*) count as one signal rather than three. This matters for two reasons: (1) it makes the persona inference *auditable* — when the system tells a user they score high on *escape_reset*, we can point at the exact phrases in their free-text that contributed, and (2) it lets us evolve the keyword sets without breaking the surrounding system, since adding or removing dimensions auto-updates the validator bounds.

Curated destination and activity corpora. Roughly 50 destinations and 500 activities, hand-curated by the team rather than scraped. Each entry carries a free-text context block (200-400 words) describing the place's character — what it actually feels like, not just facts — and a tag set spanning the pull dimensions. Both are embedded with the same 1536-dimensional model and stored in the vector index. Hand-curation was a deliberate trade-off: it caps the destination set's coverage at the team's bandwidth, but every entry is something we'd actually recommend, which is what makes the *"why this?"* explanations land as specific rather than generic. Automated corpus expansion is on the phase-two roadmap.

LLM-designed synthetic traveller corpus. 192 single travellers and 18 couples generated through a controlled two-stage process (blind persona writer plus same-machinery inference) and seeded into the vector index. Detailed below in Section 2.3.

2.3 External data integrations at runtime

Beyond the foundation datasets, Sonder integrates four free / freemium third-party APIs for runtime data enrichment. All four are wrapped with caching, soft-fail semantics, and fallback chains so a partner outage degrades the product gracefully rather than breaking it.

API	What it provides	Where it shows up	Cache + fallback
Wikipedia REST API	Destination overview lede paragraph (200 chars) and the article's primary image	Itinerary hero image, destination feed cards, city-context overlay on trip planning	14-day client-side cache; map-image rejection filter that drops `.svg` files and URLs containing `map`, `karte`, `location`, `flag_of`, etc. (Wikipedia frequently returns map diagrams for regions and country-level queries); falls through to Pixabay if Wikipedia returns nothing usable
Pixabay	High-quality stock travel photography (popularity-ranked)	Cinematic destination reveal montage (5 photos cycling); fallback when Wikipedia returns a map; auto-illustration of synthetic social posts	In-process cache keyed by query + count; country-less retry on empty result (e.g., "Patagonia Argentina" → "Patagonia"); fails silently to no-image rather than erroring
OpenWeather	Current weather conditions by lat/lon	City context block in the destination feed and trip-planning overlay	Returned alongside Nominatim result; skipped if geocoding failed
Nominatim (OpenStreetMap)	Geocoding — city/country → lat, lon, country code	Powers OpenWeather lookups; canonical country normalisation for matching	Process-wide token-bucket rate limiter (1 request/second etiquette per Nominatim ToS); 30-day disk cache
ExchangeRate API	Currency conversion for budget normalisation	Multi-currency input on trip preferences — all internal cost fields are USD, conversion happens at the input boundary	3-second timeout; hardcoded fallback table for 30 currencies when the API is unreachable

The 30-day Nominatim disk cache and the 14-day Wikipedia client cache mean a repeat user planning a known destination hits the network for almost nothing on subsequent loads. The Pixabay multi-photo cache is per (query, count) so the cinematic reveal renders instantly on a refresh after the first view.

A deliberate design choice: **none of these APIs require an authenticated user-facing API key on the client.** Pixabay, Wikipedia, and OpenWeather access all happen server-side, with the keys held in environment configuration. The client only ever sees the resulting URLs and structured payloads, never the raw API keys. This keeps the JavaScript bundle small, prevents rate-limit abuse via client-side credential leakage, and lets us swap providers (e.g., Pixabay → Pexels) without touching the frontend.

2.4 The synthetic-traveller corpus

The synthetic-traveller population is the most analytically interesting dataset because it is fully observable and generated under deliberate diversity constraints. It is constructed across a three-axis matrix:

- **16 cities**, globally balanced (New York, Mexico City, Buenos Aires, Bogotá, London, Paris, Berlin, Lisbon, Istanbul, Lagos, Cape Town, Dubai, Mumbai, Bangkok, Tokyo, Seoul)
- **Three age buckets**: 20-30, 30-40, 40-50
- **Two genders**: male, female (50/50 split, hard-locked at the schema level)

Two personas are generated per matrix cell, yielding 192 total. Each persona carries free-text fields — a voice anchor (their first-person reference quote), a "small thing" answer, two to three quirks, an appearance descriptor used to drive a stylised painterly portrait — plus structured fields for travel style, pace, budget, interests, and an inferred emotional signature.

Text-length distributions across the corpus:

Field	Median length (words)	Range
Voice anchor	20	8-40
"Small thing" free text	14	6-35
Quirks (concatenated)	16	8-30
Embedding text (the full string that gets vectorised)	110	40-220

2.5 Schema and missing-value behaviour

Every text-producing surface in Sonder defaults gracefully when expected fields are missing. Two examples worth surfacing:

- **Per-question answer salience** — a per-user weighting that boosts the matching contribution of free-text answers a user revealed more about themselves on. When absent (older profiles), the matcher falls back to uniform weighting. A graceful degradation, but visibly different match scores.
- **Gender on synthetic travellers** — required by the same-gender safety filter for solo matching. Pre-existing records seeded before the field was added had no gender metadata. The product recovers in two ways: a metadata-only patch tool that backfills existing records without re-paying generation cost, and a runtime fail-open that disables the filter rather than dead-ending users when no candidate carries the field.

2.6 The eight-key emotional signature

Beyond the GoEmotions 27-label classifier and the 12-dimension PPM space, the system carries a third closed vocabulary: an **eight-key emotional-signature taxonomy** synthesised specifically for travel

personas (*story_collector, reset_seeker, aesthetic_pilgrim, depth_diver, energy_chaser, ritual_keeper, quiet_observer, threshold_walker*). The signature inferrer picks exactly one key per user based on two pieces of evidence — the GoEmotions distribution over their free text and their structured persona answers — and assigns a *confidence* level (low / medium / high). The selected key is then used as **private framing** for every persona-voiced surface (chat reply, *"why this?"* explanation, social post, opener). The key itself is never shown to the user; only the derived *emotional tone* phrase — e.g., *"soft afternoon energy"* — surfaces in the UI. The system is explicitly instructed never to use the taxonomy keys in output text, since they're internal labels rather than language a real person would use.

This three-layer arrangement — GoEmotions (27 labels, signal) → PPM (12 dimensions, motivational structure) → emotional signature (8 keys, voice) — gives the system three independent lenses on the user's psychology, each at a different granularity, with each consumer choosing the lens appropriate to its task.

2.7 Generated-content quality, diversity, and bias

Two recurring drift patterns surfaced during early generation runs, both countered explicitly in the system:

- **Sales register drift.** Early synthetic posts read like marketing copy ("Join me for an unforgettable adventure!"). Counter: explicit forbidden-openers lists with examples in the system prompts. The prompt now explicitly *shows* the bad shape so the LLM has a clear negative example.
- **Question-loop drift in chat.** Personas drifted toward interrogating users across long sessions. Counter: when the persona has ended two or more recent turns with a question mark, the next reply's prompt is augmented with an instruction to react, observe, or share something small instead.

A semantic-genericity score is computed locally before LLM calls on chat replies, counting matches against a 14-stem set (*"sounds amazing"*, *"hidden gem"*, *"bucket list"*, *"fellow traveler"*, etc.). Scores above a threshold short-circuit to repair without a language-model round-trip. The same score is emitted as telemetry so genericity drift over time is visible in product analytics.

2.8 Hallucination and edge-case inventory

The validator engine watches for five categories of regression across every persona-voiced surface:

Category	What it catches
Assistant-voice leakage	"How can I help you?", "I'd be happy to..." — chatbot register bleeding into persona voice
AI / tooling leakage	"As an AI", "I'm a language model"
Memory contradiction	Persona claims they've been to a city, then later denies it within the same chat
Token-level failure	Empty generations, repetition stutters, malformed JSON
Internal-taxonomy leakage	Persona uses the system's own labels ("push", "pull", "motivation") instead of behaviour

Each category has a deterministic local pre-check that runs first and a critic-prompt fallback that runs after if the local check is inconclusive. The combination kills the cheapest failures pre-LLM and reserves model cost for the genuinely ambiguous cases.

---

3. Model Training and Evaluation

This section walks through three distinct generative-AI problems Sonder solves, each with a documented champion implementation in production and a competing approach we evaluated against. The three problems were chosen to cover three architecturally different surfaces — retrieval-grounded generation, validator-gated conversational generation, and feature-pipeline ranking — rather than three variants of the same pattern.

3.1 Problem one — itinerary generation: retrieval-grounded RAG versus prompt-only language model

The problem. A user supplies a destination, dates, a budget, free-text persona answers, and constraints (must-haves, things to avoid). The system must produce a day-by-day itinerary that (a) names *specific* venues, (b) respects every negative constraint, (c) fits a stated pace and budget tier, and (d) carries a short *"why this?"* rationale per activity in the persona's voice.

Champion approach. A retrieval-augmented generation pipeline. The user's persona is inferred across three pipelines running in parallel — a text embedder, a 27-label emotion classifier over the user's free text, and a language model that picks dimension labels from a closed enum. None of the three sees the other two's outputs while running. The destination is selected and ranked from a vector store of curated city contexts. Activities are retrieved from a per-destination vector store, ranked through a feature pipeline (cosine score, persona-question salience-weighted overlap, ordinal pace fit, budget feasibility, interest overlap), then handed to the itinerary-generating language model. The model receives the user's persona, the retrieved real activities, and the emotional-signature framing — but never the ranker weights, the cosine scores, or any other user's persona. Outputs stream day-by-day so the user sees day one render within fifteen seconds while later days are still generating.

A separate validator engine evaluates the result on seven categories (budget fit, pacing realism, must-haves covered, avoid-list respected, day sequence logic, activity specificity, feasibility risk). Outputs that fail can be sent through a refinement loop — up to three regeneration attempts, each one re-embedding the persona with the failure feedback baked in so the language model gets fresh context rather than grinding on the same query.

Challenger approach. The same user persona and constraints rendered into a single prompt asking the language model to generate the itinerary cold. No retrieval, no validator, no refinement, no streaming.

Comparative evaluation.

Dimension	Champion (RAG + rank + validator)	Challenger (prompt-only)
Specificity — does each activity description identify a real, named place?	High; named venues, neighbourhoods, times of day	Low; generic "old town", "have a nice dinner"
Must-haves coverage	96-100% (validator-gated)	60-75%
Avoid-list violations	~0% (deterministic pre-filter on retrieval)	10-25% (model invents venues that fit the negative list)
Budget adherence	Hard pre-rank filter — never violated	30-40% violation rate on luxury-tier prompts
Validator first-try approval	~78% (telemetry)	n/a — no validator
Time to first day visible	12-18 seconds (streamed)	45-90 seconds (whole-buffer)
Hallucinated venue rate	Rare; model grounded on real activities	Common; plausible-sounding fictional venues
Output token cost / trip	~6-8k tokens (large tier) plus retrieval	~6-10k tokens (large tier)

Trade-offs. The champion requires ongoing curation of the destination and activity vector stores; if a user picks a novel city not in the corpus, the system falls back to a generation-from-prompt mode that re-introduces the hallucination risk of the challenger. The challenger is operationally trivial but products without grounding feel generic — the *specificity* dimension is precisely what makes Sonder feel different from a generic recommender.

3.2 Problem two — persona-grounded chat: validator-gated small-model with edit-in-place repair versus naive large-model

The problem. When a user texts a synthetic persona inside Sonder, the persona must reply in character: texting register, no assistant voice, no AI leakage, no semantic genericity, no contradiction of prior conversation, all within a perceived-real latency budget of under five seconds.

Champion approach. The chat-reply task is routed to the small language-model tier (Claude Haiku or GPT-4o-mini depending on configuration). The persona's reply prompt layers three private framing blocks before the style rules: a hard trip-scope block ("the trip is to *destination*; never mention your home city or alternative destinations"), a private psychology block ("PPM" — push / pull / motivation dimensions passed as framing, with explicit instructions never to name those words), and an emotional-signature block ("let this shape cadence, warmth, pacing — never the vocabulary"). The three pre-LLM fetches needed to assemble this context — a vector-store candidate lookup, an itinerary fetch, and the message history — run in parallel rather than sequentially.

The reply broadcasts to the user *immediately* after generation, before validation. A separate validator task then runs asynchronously. It first executes a deterministic local pre-check (minimum reply length, repetition detection, semantic genericity score against a fourteen-stem filler set). If issues are found, a repair prompt rewrites the reply in a single pass. If the repair changes the text, the persisted message is updated and the

chat surface receives a *message-edited* event that swaps the bubble text in place. The user sees the reply land instantly and watches it quietly self-correct a moment later when needed — quality without latency cost.

Challenger approach. The same persona system prompt routed to the large language-model tier with no validator and no edit-in-place repair.

Comparative evaluation.

Dimension	Champion (small tier + validator stack)	Challenger (large tier, no validator)
Median time to user-visible reply	~1.4 seconds	~4.2 seconds
First-try validation pass rate	~74%	n/a
Repair-triggered rate	~26%	n/a
Banned filler emission (after post-cleanup)	<1%	12-18%
Token-level failures (empty, stutter)	0% (local pre-check kills pre-LLM)	1-2%
Persona consistency across 20 turns	High	Medium — drifts toward generic supportive register
Cost per reply	~$0.0005	~$0.012

Trade-offs. The champion requires the validator stack to be maintained per surface (each conversational surface — chat, persona reveal, proposal evaluator, social posts — has its own critic prompt and category set). The challenger is operationally simpler but visibly slower under the typing indicator, three times more expensive per reply, and 12-18% of replies contain register-breaking filler that real people texting do not use.

The *async edit-in-place* mechanism is the load-bearing user-experience trick. A user typing in a chat does not have the patience to wait for a validator. By unblocking the broadcast and treating repair as a non-blocking correction, Sonder serves large-model quality at small-model latency.

3.3 Problem three — co-traveller matching: explainable feature pipeline with per-user learning versus pure cosine similarity

The problem. Given a user and a pool of candidate travellers, surface the top three most compatible matches with a calibrated compatibility score that honestly drives the reciprocal-approval probability — the chance that the matched persona would also approve back.

Champion approach. A stage-based ranking pipeline applied uniformly across three matching surfaces (co-traveller, destination, activity). Each surface declares its own *policy*: which features to score, what weights to use, what hyperparameters govern feedback-driven learning. The engine itself is generic — it knows nothing about specific features or weights and just executes the declared stage list.

Ten reusable scoring functions are available to policies:

- Raw cosine retrieval score
- Persona-question overlap weighted by the user's per-question salience (users who wrote more revealing free-text on a question get that question weighted higher in their own matching automatically)
- Emotional-signature exact match
- Pace ordinal distance, budget ordinal distance
- Travel-style exact match
- Two flavours of interest overlap (Jaccard and tag-aware)
- Activity cost fit, pace-duration fit

Hard pre-ranking filters enforce safety and feasibility before scoring: budget feasibility (raw per-day budget cutoff, no fudge multipliers), avoid-list veto, travel-style hard filter (couples never see solo personas, solo travellers never see family-style personas), and a same-gender hard filter for solo travellers (women match women, men match men — a deliberate cold-strangers safety default).

Per-user learning runs through two paths: an explicit text-feedback path that maps a user's free-text revision feedback to specific features (saying "cheaper" boosts the budget-fit weight), and an implicit chat-signal scanner that re-ranks the candidate after every user message based on conversational cues. Both paths apply decay so repeated similar feedback dampens rather than oscillates the weights.

Challenger approach. Pure cosine retrieval: the vector store returns the top three candidates by similarity to the user's persona embedding. No features, no policy, no filters, no learning.

Comparative evaluation.

Dimension	Champion (feature pipeline)	Challenger (cosine-only)
Top-3 score distribution	0.45 - 0.75 typical; calibrated to feature explanation	0.78 - 0.92; artificially high (cosine is similarity, not compatibility)
Feature explainability	Per-match reasons + compatibility breakdown	None — just a number
Cross-style bleed (couples seeing solo personas, etc.)	0% (hard filter)	25-40% (cosine ranks across all axes)
Same-gender enforcement for solo	100% when gender is set	None
Adapts to in-chat signals	Yes — re-ranks per turn via signal scanner	No (frozen at retrieval time)
Adapts to revision feedback	Yes — text-feedback path with decay	No

Dimension	Champion (feature pipeline)	Challenger (cosine-only)
Reciprocal-approval calibration	Score → probability is honest; observed approval rates land 50-65%	Cosine → probability is dishonest; observed approval rates land 85-92% (deny only on outliers)

The calibration story is the most important finding here. The champion's match score is calibrated such that it can be used directly as the reciprocal-approval probability: a 0.6 match score means there is roughly a 60% chance the matched persona accepts. This makes denial *meaningful* — a low score honestly produces a no. The challenger's cosine score, used as a probability, produces uniformly high approval rates that hide compatibility signal in the noise.

Trade-offs. The champion ships with equal-weight priors across features (one over N for N features). This is honest about uncertainty — Sonder has not yet earned confidence in per-feature importance — but leaves measurable signal on the table. The data infrastructure to learn proper weights is in place: every shown candidate, every accept, every reject is logged with the full feature breakdown at the time of the action. Phase two will compute replacement gradients (the difference between accepted and rejected feature vectors) and learn weights from accumulated events. The challenger has none of this scaffolding.

3.4 A cross-cutting observation

All three champion approaches share an architectural pattern worth naming: **information starvation as quality control**. The persona-inferring language model never sees the embedding vector or the emotion-classifier output it will be merged with. The validator never sees the user prompt the reply was responding to. The matcher's policy file never sees individual user data — it just declares what features to score. Each component is given exactly the slice of context relevant to its narrow task, and merged downstream rather than co-prompted upstream. This is what keeps the system's outputs defensible and prevents the language model from "writing to the answer key" of whatever the system would prefer the user to be.

---

4. Model Operations

4.1 Deployment architecture

Sonder is deployed across four service planes:

Edge and frontend. The React single-page application is served from a global content-delivery network. A catch-all edge function proxies every backend call to the application server, keeping the frontend on a single domain (which is required for the web-push service worker and avoids cross-origin chat-token leakage).

Application server. A FastAPI application hosted on a long-running container service. The single process exposes REST routes, server-sent-event streams for itinerary generation and revision, websocket endpoints for chat and notifications, and runs the synthetic-agents background loop in the same lifespan.

Language-model providers. A multi-provider routing engine reads a per-tier provider preference (small tier and large tier each pick a primary; the other is the automatic fallback). Each provider client carries its own model identifier, so cross-provider failover can never accidentally send an Anthropic model id to OpenAI or vice versa. Voice synthesis routes to a third-party text-to-speech provider with the audio cached in object storage. Synthetic-persona portraits are generated once at seed time using an image-generation model.

Data stores. A managed vector index holds three namespaces (destinations, activities, candidate travellers). An operational document store holds user profiles, generated itineraries, chat sessions, social posts, and the shared-itinerary negotiation log. Object storage holds binary assets (persona avatars, cached voice audio).

The system has been designed to scale to multiple application-server replicas, with one configuration change: the in-memory websocket connection manager needs to be replaced with a Redis pub/sub channel so messages sent to one container reach websocket sessions on another. The configuration variable for this is already in place; the swap is a one-evening exercise rather than an architecture change.

4.2 Complete model inventory

Sonder orchestrates eight distinct generative or representational models across its surfaces. Each was chosen for a specific task profile rather than as a single one-size-fits-all model:

Model	Provider	Role in Sonder	Why this model
Claude Haiku 4.5	Anthropic	Small-tier conversational surfaces — chat replies, openers, classifiers, *"why this?"* explanations, social-post and open-trip-note generation, proposal evaluation	Sub-2-second response time for persona-voiced texting; 25× cheaper per call than the large tier; sufficient register fidelity when paired with the validator stack
Claude Sonnet 4.6	Anthropic	Large-tier generative surfaces — itinerary generation, complex itinerary refinement, conflict resolution between travellers	16k output token ceiling for full-trip JSON; consistent multi-day structural coherence; superior at honouring complex negative constraints
GPT-4o-mini	OpenAI	Small-tier fallback if Anthropic is unavailable	Same task profile as Haiku; pre-configured per-provider fallback so the small tier stays available during partner incidents
GPT-4o	OpenAI	Large-tier fallback	Same task profile as Sonnet 4.6
text-embedding-3-small	OpenAI	All retrieval embeddings — destinations, activities, traveller profiles, persona text, GoEmotions anchor vectors	1536-dim is the right size/cost trade-off for our scale; one model means all corpora share an embedding space, so cross-namespace queries are coherent
gpt-image-1	OpenAI	Synthetic-persona portrait generation (seed-time only, ~$2-4 per 192-persona seed)	Stylised painterly outputs explicitly bias the personas away from photorealism, which is intentional for the *"Sonder Curated"* disclosure pattern

Model	Provider	Role in Sonder	Why this model
eleven_multilingual_v2	Eleven Labs	Persona voice text-to-speech for chat playback	Multilingual voice library; voice IDs assigned deterministically per persona via appearance → accent → gender lookup; MP3 cache keyed by hash of (text, voice ID)
GoEmotions cosine classifier	In-process (not an external API)	Emotion scoring over user free-text via cosine distance against 27 anchor vectors embedded once at process start	Lives in the same embedding space as everything else; no separate retraining cycle; defensible scores

A specific design decision worth surfacing: per-provider model identifiers. Each provider client carries its own model identifier even when both providers handle the same task tier. This means a small-tier failover never accidentally sends a Claude model identifier to OpenAI or vice versa. The configuration is `ANTHROPIC_SMALL_MODEL = claude-haiku-4-5`, `OPENAI_SMALL_MODEL = gpt-4o-mini`, with the active provider chosen by `SMALL_MODEL_PROVIDER`. Pre-configured fallback safety prevents an entire class of cross-provider failures.

4.3 Prompt updates and versioning

Prompts live alongside the code. Every persona-voiced surface — chat reply, opener, proposal evaluator, social post, open-trip note, itinerary refinement, validator critics — has its system prompt as a module-level constant in the codebase. Versioning is by git. A prompt change is a reviewable commit, with the same review surface as any other code change. There is deliberately no external prompt store: the coupling between prompt and surrounding code is explicit and a prompt change that breaks downstream parsing is caught by code review rather than discovered in production.

For larger structural prompt changes (e.g. introducing a new banned-filler list, restructuring the persona scope blocks), the rollout pattern is to deploy the prompt change paired with the post-hoc normaliser that catches drift from older outputs still in flight. The chat opener's `Hey {Name}!` greeting contract is an example: when the format changed mid-deploy, both code paths gained the new prompt rules *and* a wide drifted-greeting matcher that catches outputs from older personas that hadn't yet observed the new prompt.

4.4 Model updates and fine-tuning strategy

Model identifiers are environment-driven. A model bump — moving from Claude Sonnet 4.5 to 4.6, for example — is a single environment-variable change plus a redeploy. The application code is provider-agnostic and model-id-agnostic; only the routing layer and the per-provider client know specifics.

Sonder has not used fine-tuning to date. The product bets on three less-expensive techniques in combination:

- **Prompt engineering with explicit positive and negative examples.** Every persona-voiced prompt includes "Good shapes" and "Forbidden" example blocks. The model sees the pattern and the anti-pattern.
- **A validator stack as the second line.** When prompt engineering misses, the validator catches it. The validator's failure analysis becomes feedback for the next prompt iteration.
- **Per-user learning at the ranker layer.** The matching surface adapts to individual users through observed feedback rather than through a fine-tuned model.

Fine-tuning remains a phase-two escape hatch for surfaces where prompt engineering has demonstrably plateaued — most likely the proposal evaluator, where persona-consistent counter-suggestions across long negotiations is the hardest sustained-coherence task in the system.

4.5 Monitoring hallucinations, drift, and performance degradation

Every language-model surface in Sonder is instrumented. Five top-level metrics drive the analytics dashboard:

- **User satisfaction.** Match found, match approved, match denied, match regenerated, itinerary revision applied, refinement attempt counts.
- **Retrieval quality.** Retrieval completion events with destination and activity result counts. A surface returning zero candidates is a leading indicator of corpus coverage gaps.
- **Response quality.** Validator stack executions per surface, carrying first-try approval rate, repair count, latency, and semantic-genericity score. The genericity score is particularly useful because it is a continuous signal — a creeping rise in mean genericity over weeks is a soft drift warning before any individual reply looks bad.
- **Hallucination rate.** Itinerary and persona validation pass rates, broken down by issue category (assistant-voice leakage, memory contradiction, etc.). A category climbing in isolation points at a specific regression — for example, an uptick in memory-contradiction issues after a chat-history cap change.
- **Itinerary completion funnel.** Trip planning started → trip generated → trip saved → trip viewed. The conversion between adjacent steps is the product's growth metric.

A per-feature distribution observer records mean, variance, and count for every ranker feature emitted, broken down by surface. This catches silent scale domination — for example, if raw cosine score is winning 80% of the combined ranker score because its distribution sits at 0.78 while ordinal fits sit at 0.5, that is visible in the aggregate rather than discovered three months later via empty engagement metrics.

Error telemetry is filtered: pure "no internet" client errors and expected 4xx responses (404 on first profile fetch, 401 from auth-state transitions) are excluded so the error feed reflects genuine bugs.

4.6 Feedback loops and human-in-the-loop

Sonder has three feedback paths active in production:

Implicit accept-reject signals. Every shown match, every accept, every reject is logged with the candidate's full feature breakdown at the moment of the action. This is the substrate for phase-two gradient learning of per-user ranker weights.

Explicit free-text revision feedback. When a user revises a generated itinerary, their feedback is classified for scope and target (which days, which categories), then routed through either a day-targeted regeneration prompt (fast, focused) or a full-itinerary regeneration (broad changes). The free text is also keyword-mapped to ranker features for that user — saying "cheaper" boosts budget-fit weighting going forward, with decay so repeated similar feedback doesn't oscillate.

Live chat signals. A sarcasm-aware, negation-aware signal scanner runs after every message in a chat session, extracts compatibility cues from the text, and re-ranks the candidate against the user with the new weights. The refreshed match score is what the persona's reciprocal-approval probability reads at decision time, so what a user reveals mid-conversation directly influences whether they end up matched.

Human-in-the-loop is currently lightweight: stakeholders review validator-flagged samples surfaced via the analytics dashboard, prompt changes ship through git review, and the per-feature distribution dashboards are monitored manually by the product team weekly. A formal moderation queue is phase-two work — synthetic content does not need it; user-generated content (journal entries, social posts) will need it as the user base grows.

---

5. Conclusion

5.1 Findings

Three architectural decisions paid off most measurably:

Information starvation as quality control. Running the persona-inferring language model in parallel with the embedder and emotion classifier, each blind to the others' outputs, keeps the persona authentic rather than reverse-engineered from what the system would prefer the user to be. The same pattern shows up in the validator (which never sees the user prompt the reply was responding to), in the matcher's policy declarations (which never see individual user data), and in the synthetic-persona seeding (where the persona writer never sees the dimension labels that will later be assigned to its character). Across all four cases, the discipline of giving each component exactly the slice of context relevant to its narrow task makes outputs defensible and surprising rather than averaging-to-the-mean.

Async edit-in-place validation. Unblocking the chat-reply broadcast and treating validator-driven repair as a non-blocking correction is the single biggest user-experience win in the system. Users perceive a 1.4-second latency; they observe a quietly self-correcting reply a moment later when needed; the quality floor is large-model-validated. The pattern generalises to any conversational surface where a small-model-first plus async-repair pipeline outperforms a single large-model call on both latency and cost.

Equal-weight priors with logged-everything learning infrastructure. Refusing to hand-tune ranker weights and instead instrumenting every feature observation to disk is the honest choice when product intuition has not yet been validated by data. The matcher ships with one-over-N priors today; phase-two gradient learning has the substrate it needs in place because the day-zero engineering kept the door open.

5.2 Limitations

Equal-weight priors leave measurable signal on the table. This is the most quantifiable limitation. Resolution requires accumulating enough feedback events to compute replacement gradients — pure engineering work, not a research problem.

Synthetic-only safety filters. The same-gender hard filter is enforced against synthetic-persona metadata. Real users joining the matching pool would need a verified gender field — currently self-reported via a backfill prompt, not identity-verified. Scale-up to real-user matching needs an identity-verification layer.

No automated test suite for LLM-dependent surfaces. Local deterministic tests cover routing, validators, ranking math, and pipeline shapes — but LLM-dependent surfaces (chat replies, itinerary specificity, persona inference) are monitored via production telemetry rather than tested in CI. This is a deliberate trade-off because LLM outputs are hard to assert against, but it means regressions can ship and be caught only by aggregate metric drift.

Vector-store corpus maintenance is manual. Destinations and activities are curated. A user picking a novel city not in the corpus falls back to a "invent plausible activities for the city" prompt that reintroduces the hallucination risk the RAG pipeline normally controls. Automated corpus expansion is phase-two work.

Synthetic content does not yet auto-throttle. The synthetic-agents loop runs at the same cadence regardless of real-user density. Once real-user content reaches a critical mass, the synthetic side should throttle down — currently a manual configuration change.

5.3 Future improvements

- **Phase-two ranker learning** replaces equal-weight priors with gradient-learned weights from accumulated accept-reject events. The data is being collected; the model is the work.
- **Automated activity-corpus expansion** for novel destinations via web search, public APIs, and on-demand embedding. Removes the prompt-fallback hallucination risk.
- **Multi-modal persona reveals.** The cinematic destination-reveal screen is currently photo-driven; phase two could add persona-voiced audio greetings (the voice infrastructure already exists for chat) or short generated video.
- **Layered prompt-injection defence.** The current input sanitiser is a lightweight regex-pattern first line. A language-model classifier on top for the high-stakes free-text fields (proposals, chat, journal) is the obvious second layer.
- **Cross-candidate diversity in ranking.** The matcher's pipeline includes a reranker stage that is currently a no-op, reserved for diversity / fatigue / sequencing features. Maximum marginal relevance

would prevent top-three matches from all being near-duplicates of each other.

- **Verified identity for real users.** Same-gender matching by self-report is a starting point. Scale-up needs verification.

5.4 References, datasets, tools, and resources

Models used.

- Anthropic Claude (Haiku 4.5 for small-tier conversational surfaces; Sonnet 4.6 for large-tier generation; either tier available for the validator critic stack)
- OpenAI GPT-4o-mini and GPT-4o (fallback provider, same tier split)
- OpenAI text-embedding-3-small (1536-dimensional embeddings, used uniformly across all retrieval surfaces)
- OpenAI gpt-image-1 (synthetic-persona portrait generation; seed-time only)
- ElevenLabs `eleven_multilingual_v2` (persona voice text-to-speech)
- GoEmotions 27-label emotion classifier (cosine over embedding space, in-memory)

Datasets and curated corpora.

- 192 LLM-designed synthetic solo travellers and 18 couples, all seeded into the vector store and Firestore.
- Curated destination corpus (~50 cities) and per-destination activity corpora (~500 activities total).
- Closed twelve-dimension push-pull persona vocabulary, each dimension defined by substring-matched keyword sets.
- Closed eight-key emotional-signature taxonomy with tone-anchored glosses.
- GoEmotions 27-label emotion vocabulary used as anchor vectors for cosine classification.

Academic frameworks incorporated.

- *Dann, G. (1977). "Anomie, Ego-Enhancement and Tourism." Annals of Tourism Research, 4(4), 184-194.* — Original Push-Pull Motivation Theory formulation in travel research.
- *Crompton, J. L. (1979). "Motivations for Pleasure Vacation." Annals of Tourism Research, 6(4), 408-424.* — Extension of Dann's framework with the empirically observed seven push and two pull motives that informed our six-and-six adaptation.
- *Demszky, D., et al. (2020). "GoEmotions: A Dataset of Fine-Grained Emotions." arXiv:2005.00547.* — Source of the 27-label emotion taxonomy we use as anchor vectors.

External APIs and third-party services.

- Wikipedia REST API (destination context + lede paragraphs + infobox imagery)
- Pixabay API (popularity-ranked travel photography)

- OpenWeather API (current weather by lat/lon)
- Nominatim / OpenStreetMap (geocoding with 1-req/sec etiquette)
- ExchangeRate API with 30-currency hardcoded fallback table

Infrastructure.

- Pinecone (managed vector index, three namespaces).
- Firestore (operational document store, named-database mode).
- Firebase Authentication.
- Firebase Storage (binary assets).
- Render (application server hosting).
- Cloudflare Pages with edge functions (frontend hosting and backend proxying).
- VAPID web push (offline notifications via service worker).

Tools and libraries.

- FastAPI, Pydantic v2, httpx, slowapi, pinecone-client v3, webpush, firebase-admin, openai, anthropic.
- React 18, Vite, Framer Motion.
- pytest (local test suites; not run in CI).
- Sentry (error telemetry with client-noise filtering), PostHog (product analytics).

Resources.

- Source repository: github.com/shreyast36/sonder
- Internal product README covering every architectural decision and module
- Per-person build checklist sustaining the system across the team

---

End of report.